

“Instituto Politécnico Nacional”

“Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas”



Alumno: Ortiz Campos Kevin Daniel.

Grupo: 4TM3.

Escuela: Unidad Profesional
Interdisciplinada en
Ingeniería y Tecnologías
Avanzadas.

Materia: Aplicaciones Distribuidas.

Profesor: Sierra Romero Noé.

Trabajo: Practica 04.

Practica 4 API Restful

Práctica de repaso para API Restful

Crear, probar y documentar con evidencias gráficas, los servicios Web que se solicitan.

Una vez terminados deben subirse a Git. Incluir una carpeta docs donde estarán las evidencias de funcionamiento.

Todos los servicios deben estar protegidos para evitar errores o excepciones. Y deben integrar el campo estado en la respuesta para saber si todo estuvo bien o hubo algún error.

Ejercicio 1: Servicio de Saludo Básico

// Crear un endpoint POST /saludo que reciba { "nombre": "string" }

// y responda { "estado": valor, "mensaje": "Hola, [nombre]" }

Ejercicio 2: Calculadora de Operaciones Básicas

// Endpoint POST /calcular que reciba { "a": number, "b": number, "operacion": "suma|resta|multiplicacion|division" }

// Responder { "estado": valor, "resultado": number } o { "error": "mensaje" } en caso de división por cero

Ejercicio 3: Gestor de Tareas (CRUD Básico)

// Implementar un CRUD simple para tareas:

// - POST /tareas: Crear tarea { "id": number, "titulo": string, "completada": boolean }

// - GET /tareas: Listar todas las tareas

// - PUT /tareas/:id: Actualizar una tarea

// - DELETE /tareas/:id: Eliminar una tarea // Usar un array en memoria para almacenar las tareas Agregar un campo en la respuesta: "estado": valor

Ejercicio 4: Validador de Contraseñas

// Endpoint POST /validar-password que reciba { "password": "string" }

// Validar: mínimo 8 caracteres, al menos una mayúscula, una minúscula y un número

// Responder { "estado": valor, "esValida": boolean, "errores": string[] }

Ejercicio 5: Conversor de Temperatura

// Endpoint POST /convertir-temperatura que reciba { "valor": number, "desde": "C|F|K", "hacia": "C|F|K" }

// Responder { "estado": valor, "valorOriginal": number, "valorConvertido": number, "escalaOriginal": string, "escalaConvertida": string }

Ejercicio 6: Buscador en Array

// Endpoint POST /buscar que reciba { "array": any[], "elemento": any }

// Responder { "estado": valor, "encontrado": boolean, "indice": number, "tipoElemento": string }

Ejercicio 7: Contador de Palabras

// Endpoint POST /contar-palabras que reciba { "texto": "string" }

```
// Responder { "estado": valor, "totalPalabras": number, "totalCaracteres":  
number, "palabrasUnicas": number }
```

Archivo: **Index.js**

```
const express = require('express');  
  
const app = express();  
app.use(express.json());  
  
// EJERCICIO 1: Saludo  
  
app.post('/saludo', (req, res) => {  
    const { nombre } = req.body;  
    if (!nombre || typeof nombre !== 'string') {  
        return res.status(400).json({ estado: "error", mensaje: "Nombre no  
válido" });  
    }  
    res.json({ estado: "success", mensaje: `Hola, ${nombre}` });  
});  
  
// EJERCICIO 2: Calculadora  
  
app.post('/calcular', (req, res) => {  
    try {  
        const { a, b, operacion } = req.body;  
        if (typeof a !== 'number' || typeof b !== 'number') throw new  
Error("Parámetros deben ser números");
```

```
let resultado;

switch (operacion) {

  case 'suma': resultado = a + b; break;

  case 'resta': resultado = a - b; break;

  case 'multiplicacion': resultado = a * b; break;

  case 'division':

    if (b === 0) return res.status(400).json({ estado: "error", error:
"División por cero" });

    resultado = a / b;

    break;

  default: throw new Error("Operación no válida");

}

res.json({ estado: "success", resultado });

} catch (err) {

  res.status(400).json({ estado: "error", mensaje: err.message });

}

});
```

// EJERCICIO 3: Gestor de Tareas (CRUD Básico)

```
let tareas = [];

app.post('/tareas', (req, res) => {

  const { id, titulo, completada } = req.body;
```

```
tareas.push({ id, titulo, completada });  
res.json({ estado: "success", mensaje: "Tarea creada" });  
});  
  
app.get('/tareas', (req, res) => {  
  res.json({ estado: "success", tareas });  
});  
  
app.put('/tareas/:id', (req, res) => {  
  const { id } = req.params;  
  const { titulo, completada } = req.body;  
  const index = tareas.findIndex(t => t.id === id);  
  if (index !== -1) {  
    tareas[index] = { ...tareas[index], titulo, completada };  
    return res.json({ estado: "success", mensaje: "Tarea actualizada" });  
  }  
  res.status(404).json({ estado: "error", mensaje: "No encontrado" });  
});  
  
app.delete('/tareas/:id', (req, res) => {  
  tareas = tareas.filter(t => t.id !== req.params.id);  
  res.json({ estado: "success", mensaje: "Tarea eliminada" });  
});
```

```
});
```

```
// EJERCICIO 4: Validador de contraseñas
```

```
app.post('/validar-password', (req, res) => {
```

```
  const { password } = req.body;
```

```
  let errores = [];
```

```
  if (password.length < 8) errores.push("Mínimo 8 caracteres");
```

```
  if (!/[A-Z]/.test(password)) errores.push("Falta una mayúscula");
```

```
  if (!/[a-z]/.test(password)) errores.push("Falta una minúscula");
```

```
  if (!/[0-9]/.test(password)) errores.push("Falta un número");
```

```
  res.json({
```

```
    estado: errores.length === 0 ? "success" : "error",
```

```
    esValida: errores.length === 0,
```

```
    errores
```

```
  });
```

```
});
```

```
// --- EJERCICIO 5: Conversor de Temperatura
```

```
app.post('/convertir-temperatura', (req, res) => {
```

```
  const { valor, desde, hacia } = req.body;
```

```
  let celsius;
```

```
// Conversión a Celsius
```

```
if (desde === 'C') celsius = valor;
```

```
else if (desde === 'F') celsius = (valor - 32) * 5/9;
```

```
else if (desde === 'K') celsius = valor - 273.15;
```

```
let resultado;
```

```
// Conversión de Celsius al destino
```

```
if (hacia === 'C') resultado = celsius;
```

```
else if (hacia === 'F') resultado = (celsius * 9/5) + 32;
```

```
else if (hacia === 'K') resultado = celsius + 273.15;
```

```
res.json({ estado: "success", valorOriginal: valor, valorConvertido:  
resultado, escalaOriginal: desde, escalaConvertida: hacia });  
});
```

```
// EJERCICIO 6: Buscador en array
```

```
app.post('/buscar', (req, res) => {
```

```
  const { array, elemento } = req.body;
```

```
  if (!Array.isArray(array)) return res.json({ estado: "error", mensaje: "No  
es un array" });
```

```
  const indice = array.indexOf(elemento);
```

```
  res.json({
```

```
    estado: "success",
    encontrado: indice !== -1,
    indice,
    tipoElemento: typeof elemento
  });
});

// EJERCICIO 7: Contador de Palabras
app.post('/contar-palabras', (req, res) => {
  const { texto } = req.body;

  if (typeof texto !== 'string') return res.json({ estado: "error", mensaje:
"Debe ser texto" });

  const palabras = texto.trim().split(/\s+/).filter(p => p.length > 0);
  const unicas = new Set(palabras.map(p => p.toLowerCase()));

  res.json({
    estado: "success",
    totalPalabras: palabras.length,
    totalCaracteres: texto.length,
    palabrasUnicas: unicas.size
  });
});
```

```
app.listen(3000, () => console.log('Servidor corriendo en puerto 3000'));
```

```
Administrador: Símbolo del sistema - node index.js
Microsoft Windows [Versión 10.0.22631.6199]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>cd D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful

C:\Windows\System32>D:

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful>npm init -d
npm info using npm@11.6.2
npm info using node@v24.13.0
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (proyecto4_api_restful)
version: (1.0.0)
description: Practica de repaso para API Restful
entry point: (index.js)
test command:
git repository:
keywords:
author: Daniel OC
license: (ISC)
type: (commonjs)
About to write to D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\package.json:

{
  "name": "proyecto4_api_restful",
  "version": "1.0.0",
  "description": "Practica de repaso para API Restful",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Daniel OC",
  "license": "ISC",
  "type": "commonjs"
}

Is this OK? (yes) yes

npm info ok
```

```
npm info ok

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful>npm install express

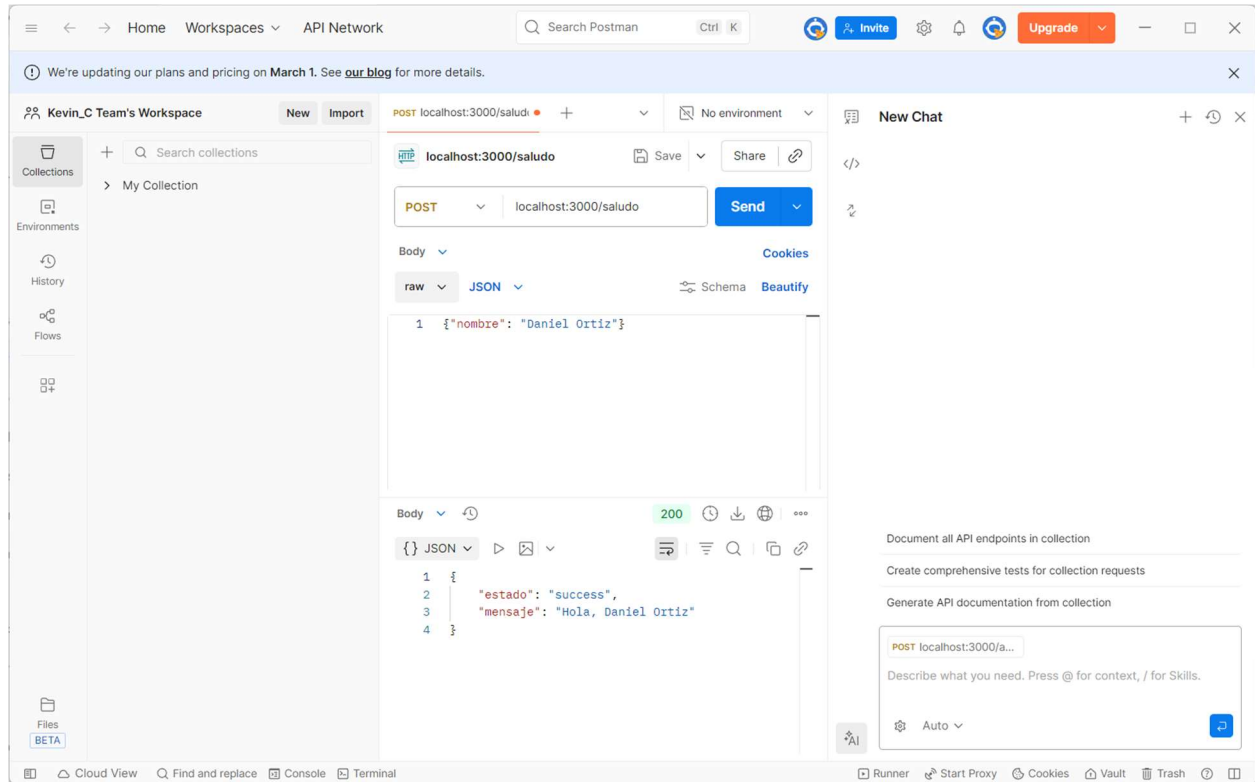
added 65 packages, and audited 66 packages in 3s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful>node index.js
Servidor corriendo en puerto 3000
```

Ejercicio 1: Servicio de Saludo Básico



Ejercicio 2: Calculadora de Operaciones Básicas

The screenshot shows the Postman interface with a workspace named "Kevin_C Team's Workspace". The main panel displays a POST request to `localhost:3000/calculador`. The request body is a JSON object: `{ "a": 14, "b": 27, "operacion": "suma" }`. The response status is 200, and the response body is a JSON object: `{ "estado": "success", "resultado": 41 }`. The right sidebar shows a "New Chat" panel with a message input field and a "Send" button.

POST localhost:3000/calculador

Body (raw): `{ "a": 14, "b": 27, "operacion": "suma" }`

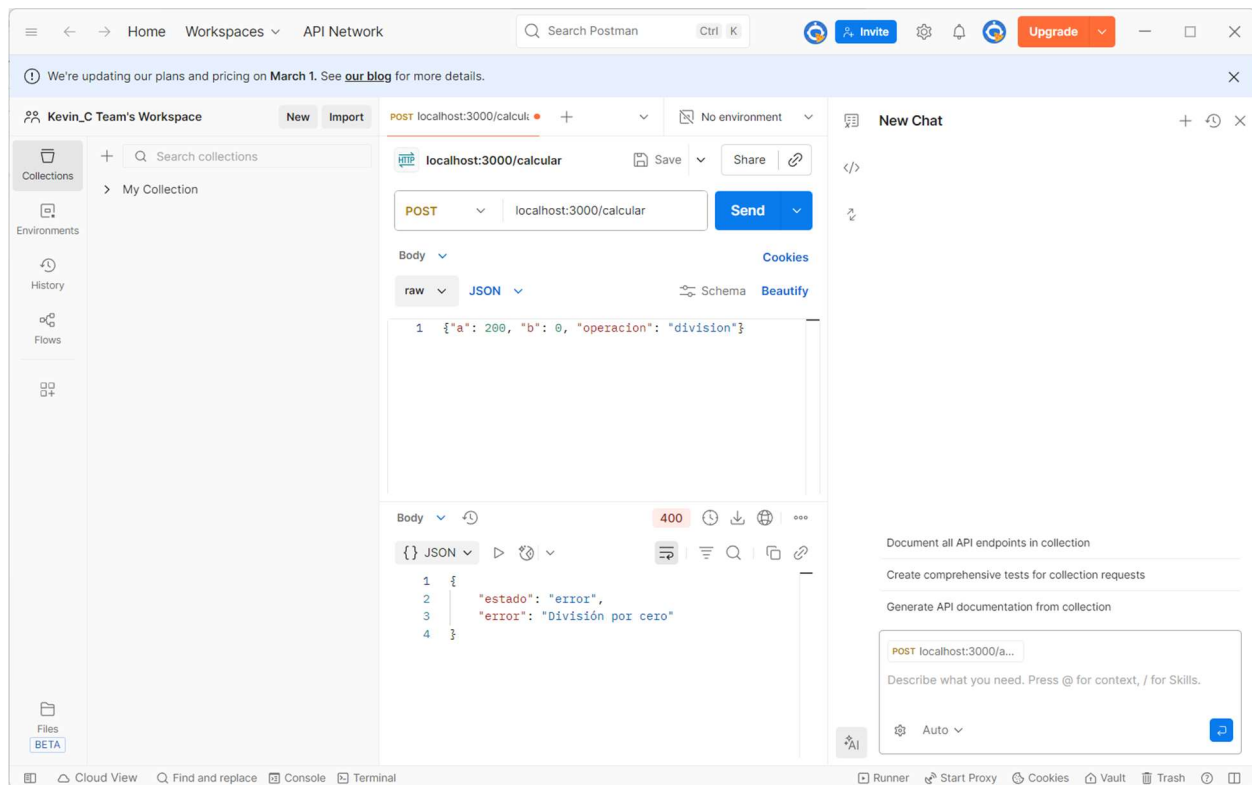
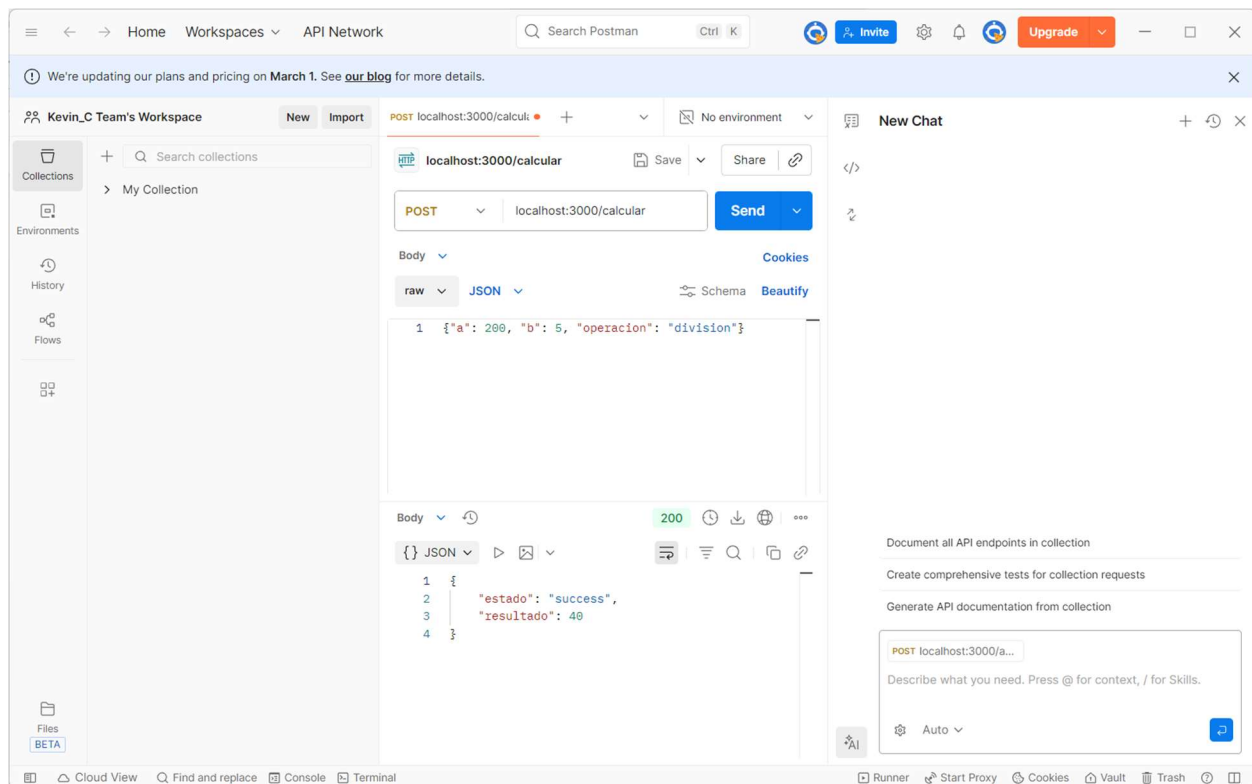
Response (JSON): `{ "estado": "success", "resultado": 41 }`

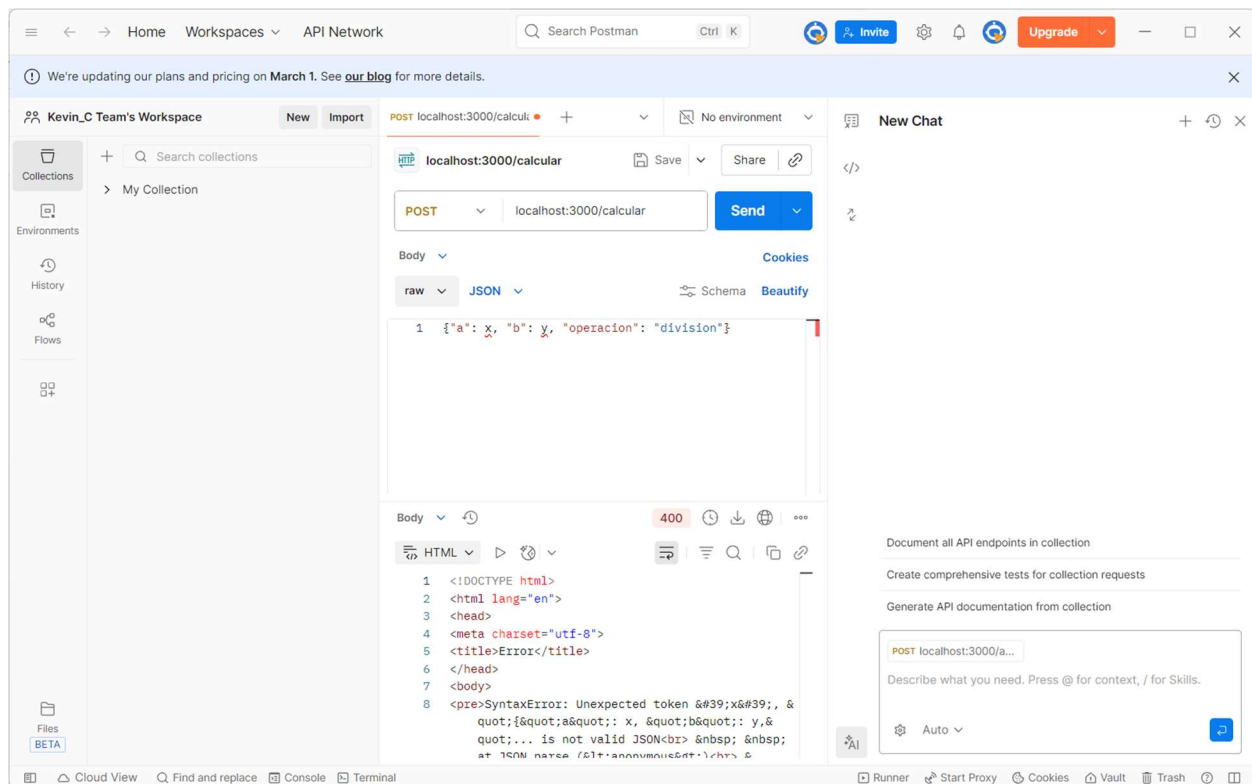
The screenshot shows the Postman interface with a workspace named "Kevin_C Team's Workspace". The main panel displays a POST request to `localhost:3000/calculador`. The request body is a JSON object: `{ "a": 23, "b": 24, "operacion": "multiplicacion" }`. The response status is 200, and the response body is a JSON object: `{ "estado": "success", "resultado": 552 }`. The right sidebar shows a "New Chat" panel with a message input field and a "Send" button.

POST localhost:3000/calculador

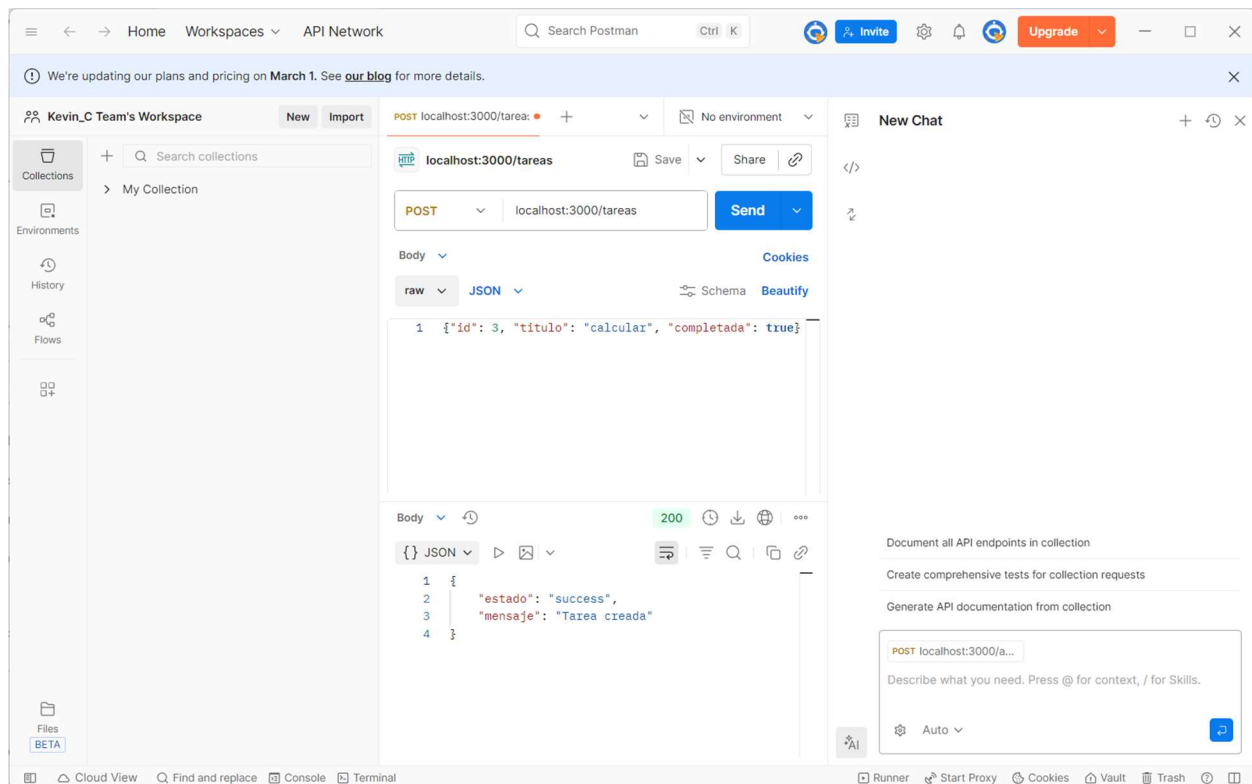
Body (raw): `{ "a": 23, "b": 24, "operacion": "multiplicacion" }`

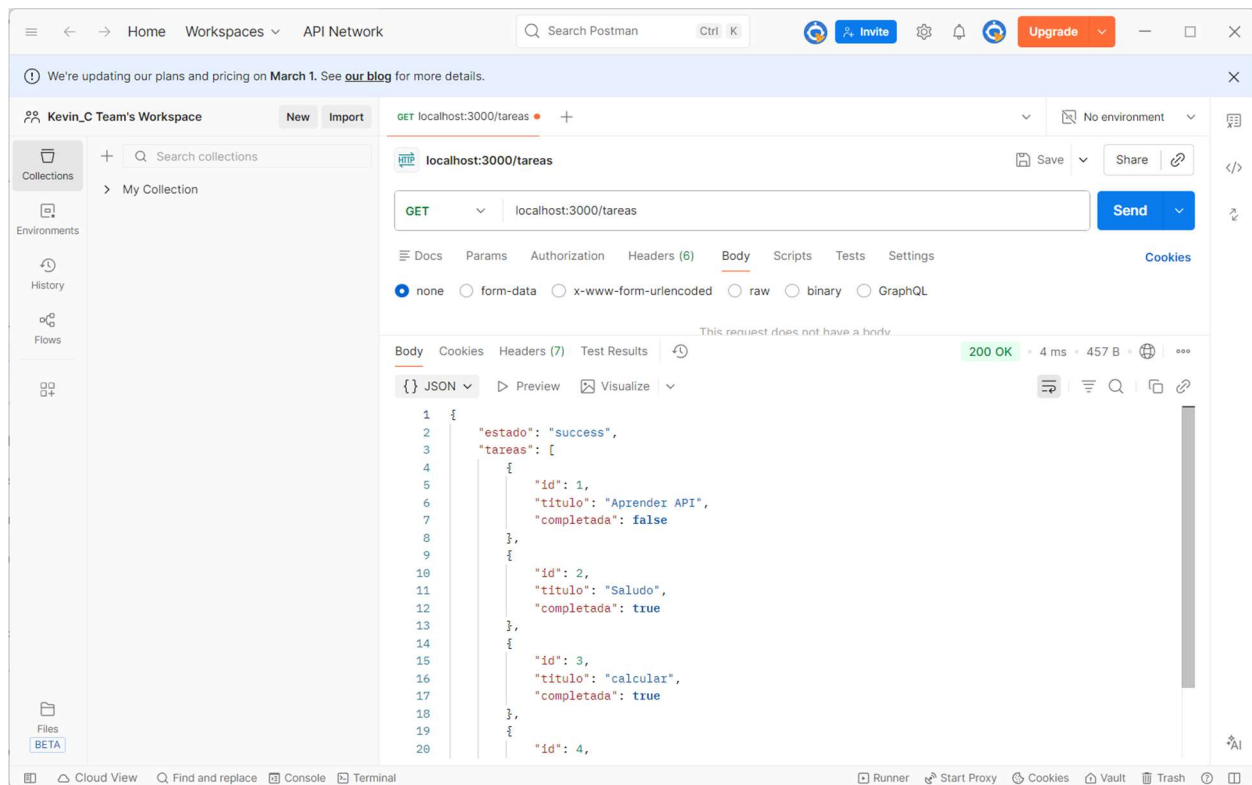
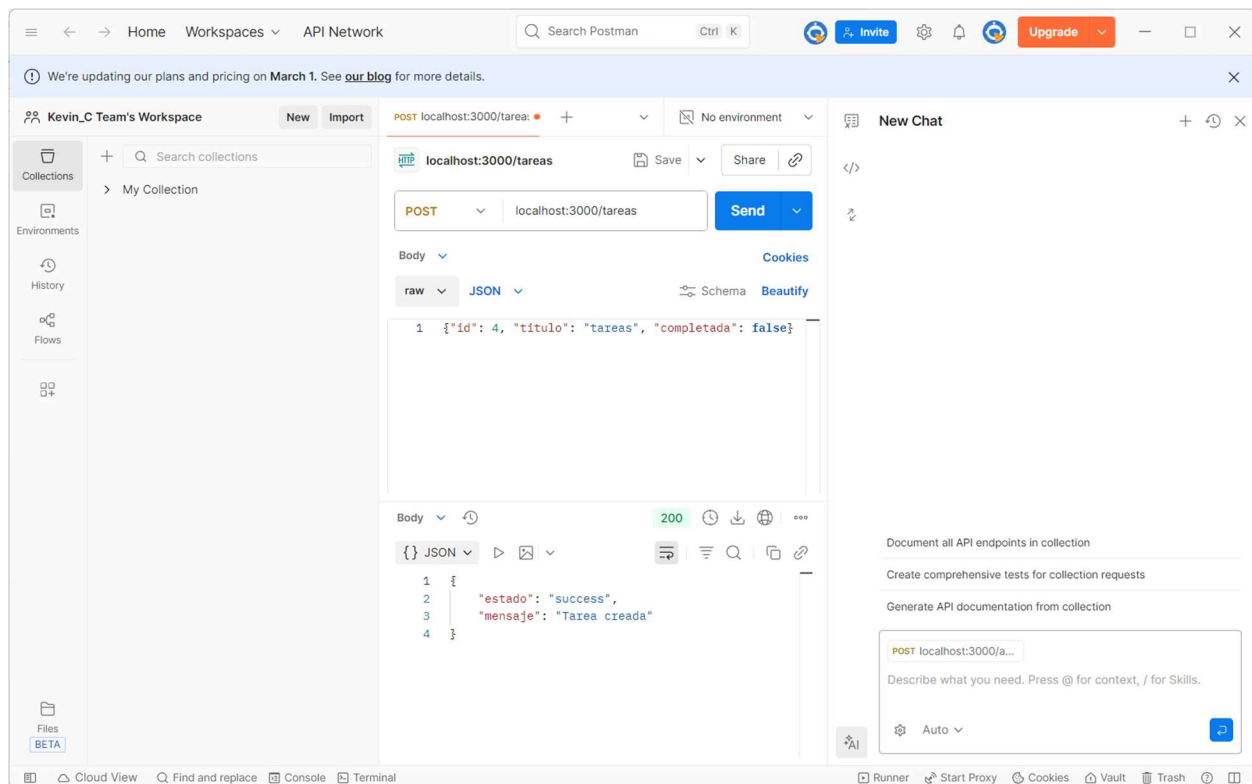
Response (JSON): `{ "estado": "success", "resultado": 552 }`

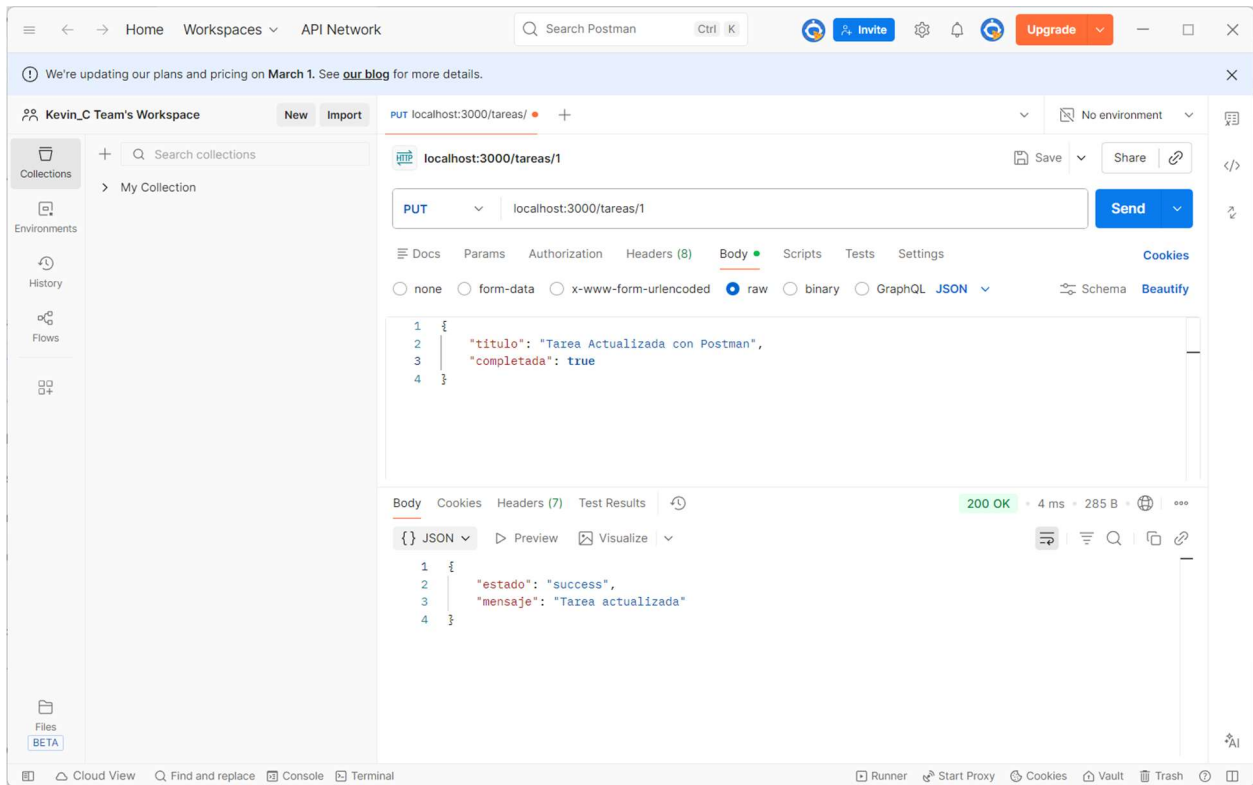
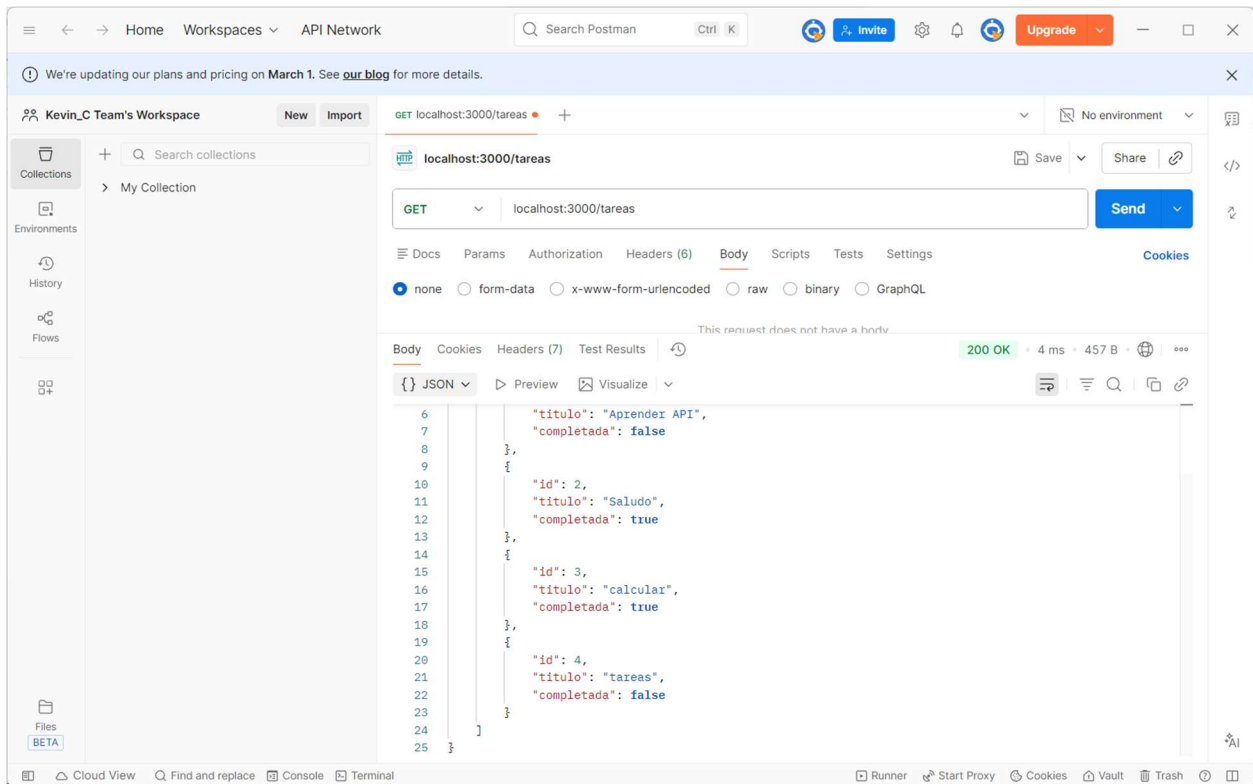


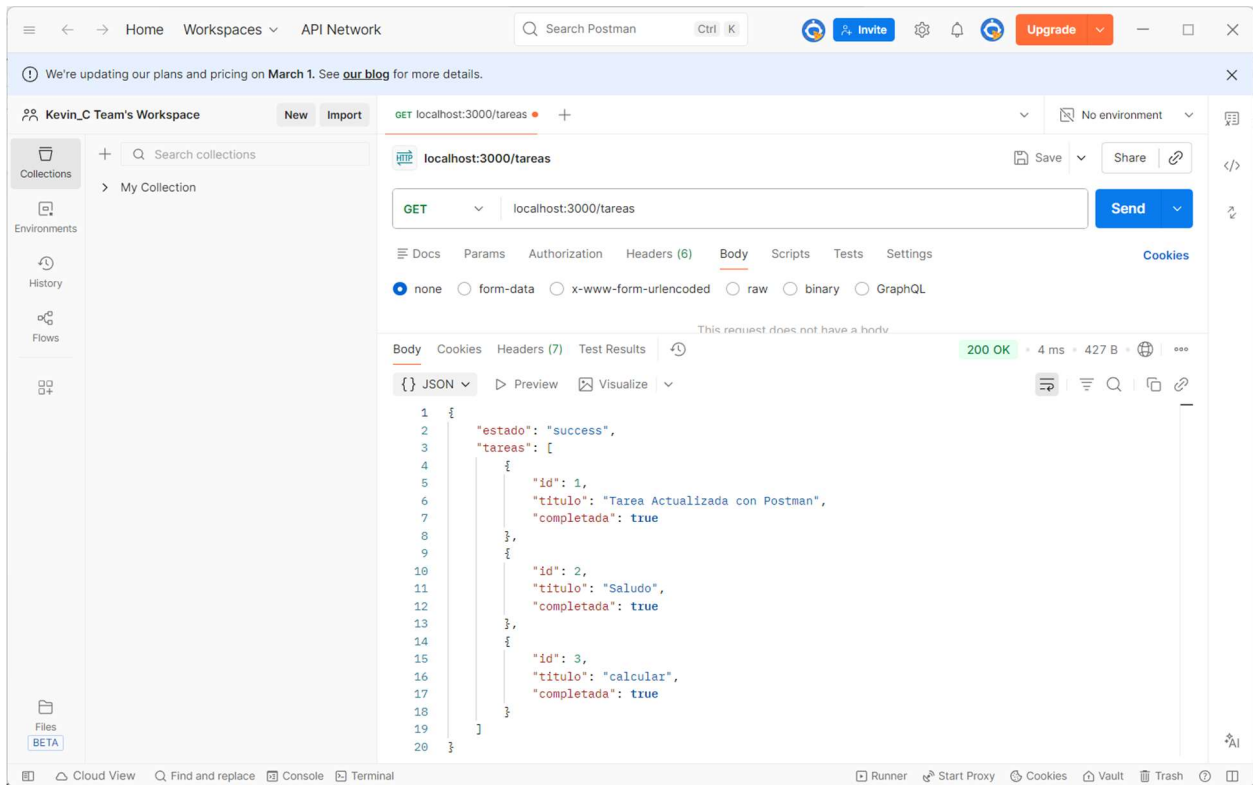
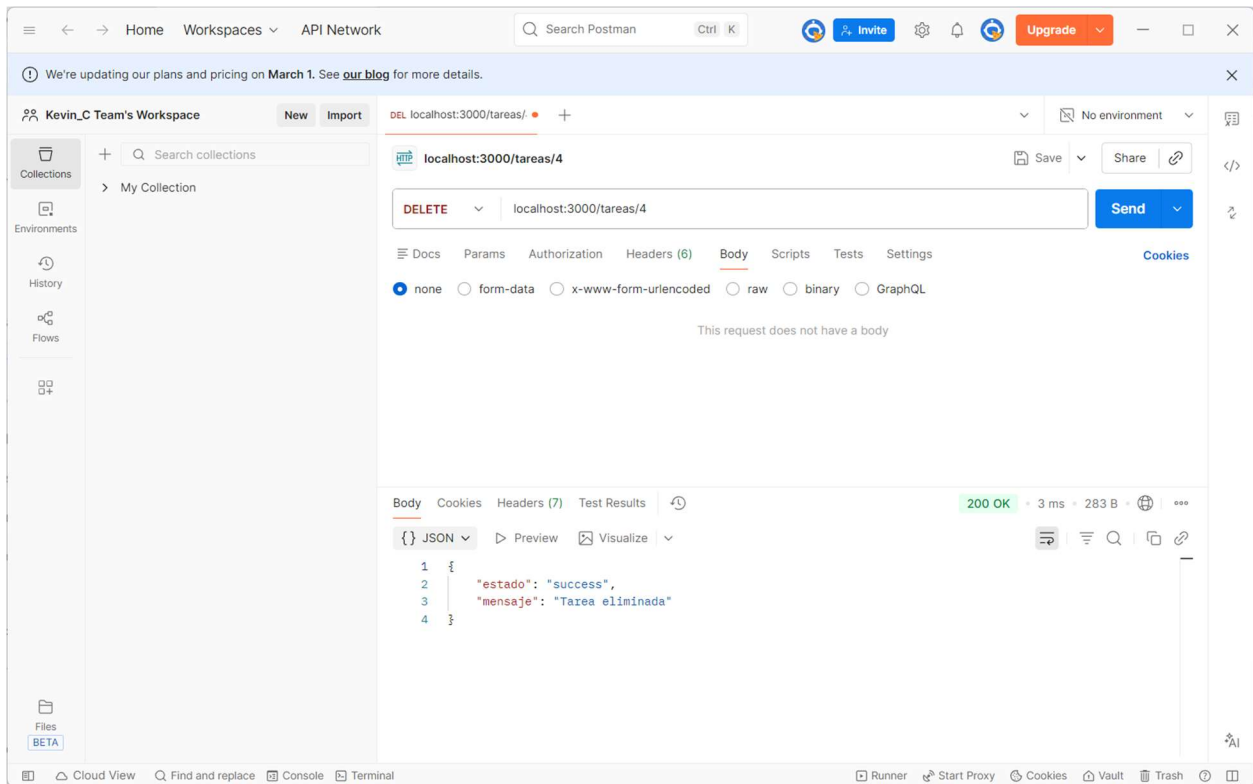


Ejercicio 3: Gestor de Tareas (CRUD Básico)









Ejercicio 4: Validador de Contraseñas

The screenshot shows the Postman application interface. The top bar includes navigation links (Home, Workspaces, API Network), a search bar, and utility buttons (Invite, Upgrade). A notification banner at the top states: "We're updating our plans and pricing on March 1. See [our blog](#) for more details."

The main workspace is titled "Kevin_C Team's Workspace" and contains a single collection named "My Collection". The selected request is a POST to "localhost:3000/validar-password". The request body is set to "raw" (JSON) with the payload:

```
1 {"password": "Pass12345"}
```

The response is displayed in the "Body" tab, showing a 200 OK status with a response time of 4 ms and a size of 284 B. The JSON response is:

```
1 {
2   "estado": "success",
3   "esValida": true,
4   "errores": []
5 }
```

The bottom status bar includes icons for Cloud View, Find and replace, Console, Terminal, Runner, Start Proxy, Cookies, Vault, Trash, and a help icon.

Ejercicio 5: Conversor de Temperatura

The screenshot displays the Postman application interface. The top navigation bar includes 'Home', 'Workspaces', and 'API Network'. A search bar and a 'Search Postman' button are present. A notification banner at the top states: 'We're updating our plans and pricing on March 1. See [our blog](#) for more details.'

The main workspace is titled 'Kevin_C Team's Workspace'. On the left sidebar, there are sections for 'Collections' (with a search bar and 'My Collection'), 'Environments' (with 'No environment' selected), 'History', 'Flows', and 'Files' (with a 'BETA' badge).

The central area shows a POST request to 'localhost:3000/convertir-temperatura'. The request body is raw JSON:

```
1 {"valor": 100, "desde": "C", "hacia": "F"}
```

. The response is a 200 OK status with a 3 ms response time and 342 B of data. The response body is JSON:

```
1 {
2   "estado": "success",
3   "valorOriginal": 100,
4   "valorConvertido": 212,
5   "escalaOriginal": "C",
6   "escalaConvertida": "F"
7 }
```

The bottom status bar includes icons for 'Cloud View', 'Find and replace', 'Console', 'Terminal', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.

Ejercicio 6: Buscador en Array

The screenshot shows the Postman interface for a workspace named "Kevin_C Team's Workspace". A POST request is configured to "localhost:3000/buscar". The request body is set to "raw" and contains the following JSON:

```
1 {"array": [10, 20, 30], "elemento": 20}
```

The response is a 200 OK status with a response time of 3 ms and a body size of 308 B. The response body is displayed in JSON format:

```
1 {
2   "estado": "success",
3   "encontrado": true,
4   "indice": 1,
5   "tipoElemento": "number"
6 }
```

The screenshot shows the Postman interface for the same workspace. A POST request is configured to "localhost:3000/buscar". The request body is set to "raw" and contains the following JSON:

```
1 {"array": [11, 26, 37], "elemento": 20}
```

The response is a 200 OK status with a response time of 9 ms and a body size of 310 B. The response body is displayed in JSON format:

```
1 {
2   "estado": "success",
3   "encontrado": false,
4   "indice": -1,
5   "tipoElemento": "number"
6 }
```

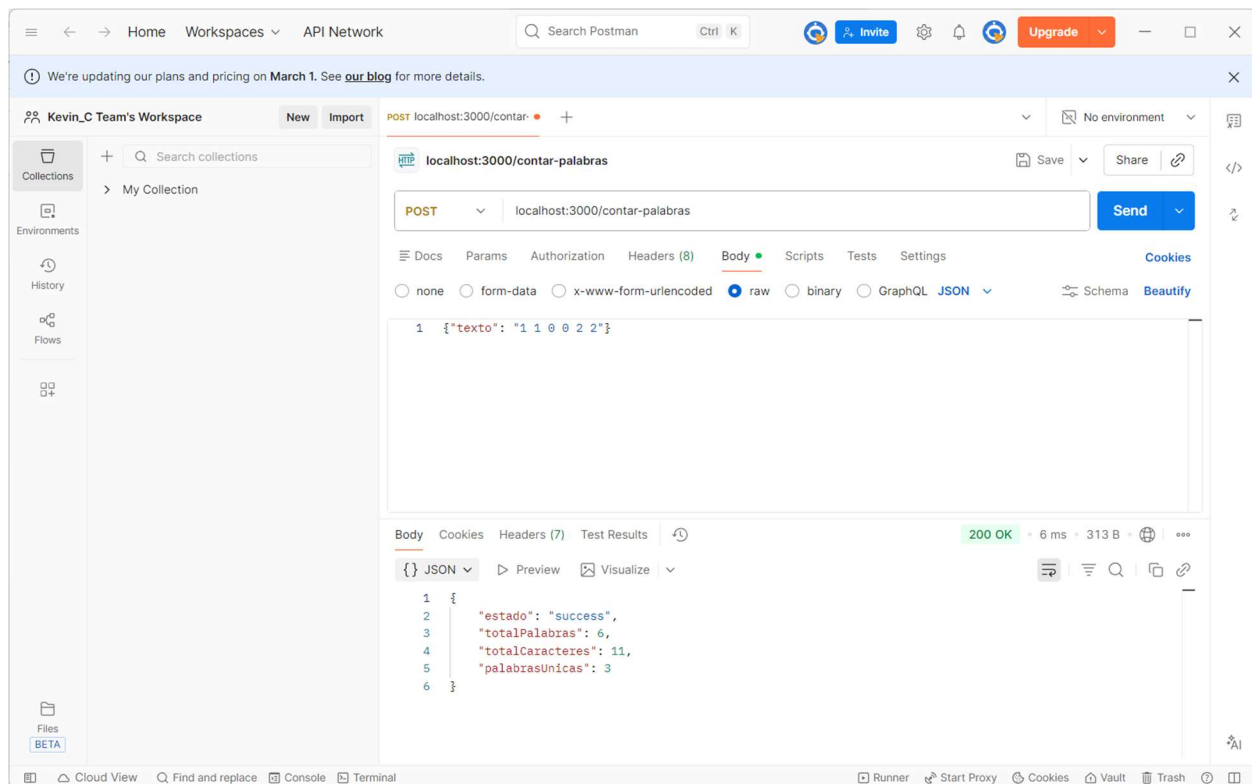
Ejercicio 7: Contador de Palabras

The screenshot shows the Postman interface with a workspace named "Kevin_C Team's Workspace". A POST request is configured to "localhost:3000/contar-palabras". The request body is raw text: `{ "texto": "Hola mundo !" }`. The response is a 200 OK status with a JSON body:

```
1 {
2   "estado": "success",
3   "totalPalabras": 3,
4   "totalCaracteres": 12,
5   "palabrasUnicas": 3
6 }
```

The screenshot shows the Postman interface with the same workspace. The POST request body is now a longer string: `{ "texto": "Mi nombre es Máximo Décimo Meridio, comandante de los ejércitos del Norte, general de las legiones Félix, leal servidor del verdadero emperador Marco Aurelio, padre de un hijo asesinado, marido de una esposa asesinada, y alcanzaré mi venganza, en esta vida o en la otra" }`. The response is a 200 OK status with a JSON body:

```
1 {
2   "estado": "success",
3   "totalPalabras": 45,
4   "totalCaracteres": 269,
5   "palabrasUnicas": 39
6 }
```



```
D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful>node index.js
Servidor corriendo en puerto 3000
SyntaxError: Unexpected token 'x', "{a": x, "b": y,"... is not valid JSON
    at JSON.parse (<anonymous>)
    at parse (D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\node_modules\body-parser\lib\types\json.js:72:19)
    at D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\node_modules\body-parser\lib\read.js:162:18
    at AsyncResource.runInAsyncScope (node:async_hooks:214:14)
    at invokeCallback (D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\node_modules\raw-body\index.js:238:16)
    at done (D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\node_modules\raw-body\index.js:227:7)
    at IncomingMessage.onEnd (D:\Documents\UNI papeles\11º Semestre\Aplicaciones Distribuidas\Proyectos\Proyecto4_API_Restful\node_modules\raw-body\index.js:287:7)
    at IncomingMessage.emit (node:events:508:28)
    at endReadableNT (node:internal/streams/readable:1701:12)
    at process.processTicksAndRejections (node:internal/process/task_queues:89:21)
```